

Syllabus : Exploitation des grands modèles linguistiques (LLM) pour la détection de vulnérabilités en Cybersécurité

Auteur : Badr TAJINI - LLM pour la détection de vulnérabilités en Cybersécurité - ECE - 2025/2026

CREDITS ECTS : X

NOMBRE D'HEURES DISPENSEES : 16H

COURS MAGISTRAUX	TRAVAUX DIRIGES	TRAVAUX PRATIQUES	PROJETS
8	8	0	0

NOMBRE D'HEURES DE TRAVAIL PERSONNEL : ~20h pour le projet final

MODALITE PEDAGOGIQUE : PRESENTIEL

LANGUE : FRANÇAIS / ANGLAIS (interactions LLM et ressources souvent en anglais)

Objectifs

- Les objectifs du cours sont les suivants :
 - Comprendre le rôle et le potentiel des LLM en cybersécurité.
 - Maîtriser les concepts fondamentaux des LLM pertinents pour la sécurité.
 - Développer des compétences en ingénierie des prompts pour la détection de vulnérabilités.
 - Appliquer les LLM à l'analyse de code source et de configurations pour identifier des failles potentielles.
 - Évaluer de manière critique les résultats des LLM et comprendre leurs limites intrinsèques.
 - Utiliser les LLM de manière éthique et responsable dans un contexte de sécurité (confidentialité, biais).

Prérequis

- Connaissances fondamentales en cybersécurité (types de vulnérabilités communes, ex: OWASP Top 10).
- Concepts de base en programmation / scripting (lecture et compréhension de code - ex: Python, JavaScript, Java).
- Intérêt pour l'intelligence artificielle et ses applications en sécurité.

Contenu

Le cours comprend les séances théoriques suivantes (8 modules d'environ 1h) :

1. Module 1 : Introduction - IA & LLM dans le paysage de la cybersécurité
2. Module 2 : Fondamentaux des LLM pour les ingénieurs en cybersécurité
3. Module 3 : L'Art de l'ingénierie des prompts pour la détection de vulnérabilités

4. Module 4 : Application des LLM à l'analyse de code source (Amélioration SAST)
5. Module 5 : LLM pour la revue de configurations & la sécurité IaC (Infrastructure as Code)
6. Module 6 : Au-delà du code & des configurations - autres applications potentielles
7. Module 7 : Évaluation des résultats des LLM & l'impératif de l'humain dans la boucle
8. Module 8 : Limitations, risques de sécurité liés aux LLM, et tendances futures

*Note : Les 8h de mini-projets (4 mini-projets * 2h) seront consacrées à la mise en pratique de ces concepts (prise en main des LLM, prompting pour code/config, analyse critique).*

Compétences à acquérir

1. Expliquer les capacités et les limitations des LLM pour les tâches d'analyse de sécurité.
2. Formuler des prompts (requêtes) clairs, spécifiques et contextuels pour interroger efficacement un LLM sur des questions de cybersécurité.
3. Utiliser un LLM comme assistant pour identifier des vulnérabilités potentielles dans des extraits de code (ex: Python, JavaScript, Java).
4. Utiliser un LLM pour rechercher des erreurs de configuration courantes dans des fichiers de configuration (ex: SSH, Nginx, politiques Cloud).
5. Analyser de manière critique et vérifier les suggestions de sécurité ou les vulnérabilités signalées par un LLM.
6. Comprendre les risques liés à la confidentialité des données lors de l'utilisation de LLM avec du code ou des configurations sensibles.
7. Rédiger un rapport de revue de sécurité succinct basé sur les résultats d'un LLM, incluant une analyse humaine critique.

Modalités d'évaluation

- Contrôle Continu (mini-projets notés) - **50%**
- Projet Final (Réalisation d'une revue de sécurité assistée par LLM sur un artefact fourni, avec rapport détaillé - analyser/critiquer des interactions LLM) - **50%**

Références bibliographiques :

- [1] **Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022).** *Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions.* In IEEE Symposium on Security and Privacy (SP) 2022.
- [2] **Noever, D. (2023).** *Can Large Language Models Find And Fix Vulnerabilities? A Case Study on GitHub Copilot.* arXiv preprint arXiv:2304.01467.
- [3] **Fu, T., Chen, J., Li, W., Wang, H., Jin, H., & Zhang, H. (2023).** *Does GPT-4 Pass the Bar Exam on Cybersecurity? Evaluating GPT-4's Security Capabilities.* arXiv preprint arXiv:2312.11341.
- [4] **Sobania, D., Briesch, M., Hanna, C., & Petke, J. (2023).** *An Analysis of the Automatic Bug Fixing Performance of ChatGPT.* arXiv preprint arXiv:2301.08653.
- [5] **Schuster, E., Adolphs, L., Lammich, P., & Neumann, G. (2023).** *Can we trust LLMs? An Assessment of Code Generation Capabilities of ChatGPT.* arXiv preprint arXiv:2307.08164.
- [6] **Fischer, M., Zipf, S., & Krombholz, K. (2024).** *Large Language Models for Vulnerability Detection: A Systematic Literature Review.* arXiv preprint arXiv:2402.15416.

- [7] **Oliyunkov, R. (2023).** *Large Language Models for Source Code Analysis*. Vector Institute.
- [8] **Perry, N., Simko, M., & Aharoni, M. (2023).** *Security Implications of Large Language Models*. IBM Research.
- [9] **Happe, A., & Siemens Cybersecurity Team. (2023).** *Using Large Language Models (LLMs) for cybersecurity: perspectives, benefits, and risks*. Siemens Whitepaper.
- [10] **Kang, D., Li, X., Sims, C., Appel, A. W., & Wang, T. (2023).** *Exploring the Security Vulnerabilities of Large Language Models*. arXiv preprint arXiv:2311.12513.
- [11] **Jones, N. (2023).** *How AI is shaking up code generation — and cybersecurity*. Nature News Feature.
- [12] **Dakhel, A. M., Rauf, U., Eltanbouly, S., Malik, H., & Nandi, A. (2023).** *Leveraging Large Language Models for Automated Unit Test Generation and Vulnerability Detection*. In Proceedings of the 2023 ACM/IEEE International Conference on Automation of Software Test (AST).

Ressources supplémentaires :

- **OWASP Top 10 for Large Language Model Applications:** Projet OWASP dédié aux risques spécifiques des applications basées sur les LLM. Utile pour comprendre les vecteurs d'attaque contre les LLM eux-mêmes. (<https://owasp.org/www-project-top-10-for-large-language-model-applications/>)
 - **Blogs Techniques des Fournisseurs de LLM (OpenAI, Google AI, Anthropic) et Entreprises de Sécurité (NCC Group, Trail of Bits, Synack, etc.) :** Publient souvent des articles sur les capacités, limitations et utilisations sécuritaires de leurs modèles ou des LLM en général.
-

Annexe :

Feuille de route du cours - Enseignement magistral :

1. Module 1 : Introduction - IA & LLM dans le paysage de la cybersécurité (1h)

- Le rôle évolutif de l'IA en cybersécurité (détection, réponse, renseignement sur les menaces).
- Que sont les LLM ? Aperçu de haut niveau (Transformers, Tokens, Données d'entraînement). Pas de mathématiques approfondies, focus sur la compréhension conceptuelle pertinente pour la sécurité.
- Pourquoi utiliser les LLM pour la détection de vulnérabilités ? Bénéfices potentiels (vitesse, échelle, nouvelles perspectives).
- Opportunités vs. Hype/Engouement : définir des attentes réalistes.
- Considérations éthiques & utilisation responsable : confidentialité des données (critique lors de la saisie de code/configurations), biais, utilisation abusive potentielle. Aperçu de la structure et des objectifs du cours.

2. Module 2 : Fondamentaux des LLM pour les ingénieurs en cybersécurité (1h)

- Concepts clés des LLM : prompting (formulation de requêtes), fenêtre de contexte, température, tokens, embeddings (simplifié).
- Différents types/accès aux LLM : modèles généralistes (séries GPT, Claude, Llama, Bard) vs. modèles potentiellement spécialisés en sécurité (si disponibles/pertinents). Accès API vs. Interface Web.

- Forces dans le contexte de la sécurité : compréhension du langage naturel, compréhension du code (à des degrés divers), reconnaissance de motifs, résumé.
- Faiblesses & risques : hallucinations (faux positifs/négatifs), manque de véritable compréhension/raisonnement, vulnérabilité à l'injection de prompts, empoisonnement des données, dépendance à la qualité et à la fraîcheur des données d'entraînement.

3. Module 3 : L'Art de l'ingénierie des prompts pour la détection de vulnérabilités (1.5h)

- Principes d'un prompting efficient : clarté, contexte, spécificité, jeu de rôle ("Agis comme un auditeur de code de sécurité...").
- Prompting Zero-Shot vs. Few-Shot : fournir des exemples dans le prompt.
- Structuration des prompts pour les tâches de sécurité :
 - Saisir efficacement des extraits de code/fichiers.
 - Demander des types de vulnérabilités spécifiques (ex: "Vérifie ce code Python pour d'éventuelles vulnérabilités d'injection SQL").
 - Demander des explications, des évaluations de sévérité (avec prudence), ou des conseils de remédiation potentiels.
 - Formatage des demandes de sortie (ex: "Liste les résultats avec les identifiants CWE").
- Raffinement itératif des prompts : comment améliorer les prompts en fonction des réponses du LLM.

4. Module 4 : Application des LLM à l'analyse de code source (Amélioration SAST) (1.5h)

- Utiliser les LLM pour identifier les vulnérabilités de code courantes (focus sur le Top 10 OWASP : SQLi, XSS, Désérialisation Insécurisée, Injection de Commandes, etc.).
- Spécificité du langage de programmation : Comment la performance des LLM varie selon les langages (ex: Python, Java, JavaScript, C#).
- Comparaison avec les outils SAST traditionnels : forces (requêtes en langage naturel, trouver des failles logiques ?) et faiblesses (cohérence, complétude, faux positifs/négatifs, défis de scalabilité ?).
- Études de cas/exemples : démonstration d'identification réussie (et infructueuse) de vulnérabilités dans des extraits de code.

5. Module 5 : LLM pour la revue de configurations & la sécurité IaC (1h)

- Analyse des fichiers de configuration pour les erreurs courantes (ex: serveurs web, bases de données, SSH, services cloud - IAM, buckets S3).
- Identifier les paramètres par défaut non sécurisés, les réglages trop permissifs, les contrôles de sécurité manquants.
- Application à l'Infrastructure en tant que Code (IaC) : Examiner Terraform, CloudFormation, etc., pour les failles de sécurité avant le déploiement.
- Stratégies de prompting spécifiques à l'analyse de configuration.

6. Module 6 : Au-delà du code & des configurations - autres applications potentielles (0.5h)

- Analyse de la documentation de sécurité, des politiques, des rapports de renseignement sur les menaces (résumé, extraction d'entités).
- Aide à l'interprétation des résultats de scans de vulnérabilités ou des découvertes de pentest.

- *Note de mise en garde* : Souligner que les limitations et les risques augmentent à mesure que la tâche devient moins structurée que la revue de code/config. Potentiel d'analyse de descriptions textuelles de trafic réseau ou de logs (aperçus de haut niveau, pas d'analyse de paquets profonde).

7. Module 7 : Évaluation des résultats des LLM & l'impératif de l'humain dans la boucle (1h)

- Techniques de vérification des découvertes du LLM : revue manuelle, recoupement avec d'autres outils, tests ciblés (si applicable).
- Comprendre et gérer les faux positifs & faux négatifs.
- Évaluer la confiance : Les LLM semblent souvent confiants même lorsqu'ils ont tort. Pourquoi l'expertise humaine reste essentielle.
- Construire un flux de travail qui intègre les LLM comme *assistants*, et non comme remplaçants. Contrer la dépendance excessive.

8. Module 8 : Récapitulatif des limitations, risques de sécurité liés aux LLM, et tendances futures (0.5h)

- Récapitulatif des limitations clés (hallucinations, fenêtres de contexte, date limite des données d'entraînement, confidentialité).
- Risques de sécurité *pour l'utilisateur* : Injection de prompt (fuite d'informations sensibles dans les prompts), violations de la confidentialité des données.
- Attaques adverses *contre* les LLM (bref aperçu).
- Perspectives d'avenir : LLM spécialisés en sécurité, intégration dans les IDEs et plateformes de sécurité (ex: fonctionnalités de sécurité de GitHub Copilot), recherche en cours dans la communauté.